Authentication and Authorization in Practice

From password cracking and passkeys to access control models and real-world breaches

Diellart Mulolli (230756200035)

Code and LaTeX source: https://github.com/Diellart-Mulolli/auth_lab

October 27, 2026

Introduction

Before a system lets anyone act, it has to answer two questions: *who are you* (authentication) and *what may you do* (authorization). Passwords, biometrics, one-time codes and passkeys answer the first question; access control matrices, roles and multilevel security labels answer the second. Attackers usually go after these two layers rather than the cryptography underneath, because a stolen password, a tricked approval prompt or an over-broad permission is far cheaper than breaking an algorithm. This report works through the mathematics of password cracking and biometric error rates, compares one-time codes with phishing-resistant passkeys, analyses an access control matrix and its confused-deputy weakness, and applies the Bell–LaPadula and Biba rules to concrete requests. It then investigates the 2019 Capital One cloud breach as a real failure of authorization. Six small Python scripts were used to verify every calculation and simulation; they are published, with the LaTeX source and a running guide, at https://github.com/Diellart-Mulolli/auth_lab. Appendix A lists the values chosen in each script and its exact output.

Authentication

Question 1 — The math of password cracking, and what a good password policy looks like

Parameters: $N = 64^{10} = 2^{60}$ possible passwords, $U = 2^{12}$ users, a dictionary of $D = 2^{24}$ words, and each password is in the dictionary with probability $p = 1/8$. Work is counted in hash computations (Stamp, ch. 6 [1]); comparisons are free. All values below were computed by `q1_password_cracking.py` (Appendix A.1).

(a) Expected work of Trudy in four cases.

Case		Unsalted	Salted
I	Alice's password, no dictionary	2^{59}	2^{59}
II	Alice's password, with dictionary	$\approx 2^{59}$ ($2^{58.8}$; the dictionary phase costs $2^{23.9}$ and works only with probability 1/8)	same as unsalted
III	any password, no dictionary	$2^{60}/(2^{12}+1) \approx 2^{48}$	2^{59}
IV	any password, with dictionary	$D/(Up+1) \approx 2^{24}/2^{9} = 2^{15}$	$D/p = 2^{24} \cdot 8 = 2^{27}$

How each entry arises:

- *Case I.* Brute force over 2^{60} passwords finds Alice's on average after half of them, 2^{59}. A salt changes nothing: it is stored in the clear, so Trudy hashes every guess once with Alice's salt.

- *Case II.* Trudy tries the 2^{24} words first. This succeeds only with probability 1/8 (on average after 2^{23} words); in the other 7/8 of the cases she must still brute-force 2^{59}. The expected total is $\frac{1}{8}2^{23} + \frac{7}{8}(2^{24} + 2^{59}) \approx 2^{58.8}$, so against one chosen victim a dictionary barely helps.

- *Case III.* Without salt every guess can be compared (for free) with all 2^{12} stored hashes, so the first of U passwords falls after $N/(U+1) \approx 2^{48}$ guesses. With salt a guess hashed with one user's salt is comparable only with that user's hash, so attacking everyone at once gives no speed-up: 2^{59}.

- *Case IV.* Unsalted, Trudy hashes the dictionary once and compares with every entry; about $Up = 2^9$ users are in the dictionary, so the first hit appears after about $D/(Up+1) \approx 2^{15}$ words (and the whole file is cracked after at most 2^{24}). Salted, she must scan the dictionary again for each user; each user fails with probability 7/8 and costs D, so the expected work is $\frac{1-p}{p}D + \frac{D}{2} \approx 2^{26.9} \approx D/p = 2^{27}$, the rule of thumb of the lab handout.

The closed forms for III and IV were also checked by a Monte Carlo simulation of the same model at smaller sizes (agreement within 2%, Appendix A.1).

(b) For Case IV with salted passwords ($2^{27} = 134\,217\,728$ hashes): at 10^{10} hashes/s (fast hash such as SHA-256) the work takes $1.34 \cdot 10^8/10^{10} \approx$ **13.4 ms**; at 10^4 hashes/s (memory-hard Argon2id) it takes $1.34 \cdot 10^8/10^4 \approx 1.34 \cdot 10^4$ s $\approx$ **3.7 hours** (the exact expectation gives 12.6 ms and 3.5 hours).

(c) Salting multiplies Trudy's work by about the number of users in both Case III and Case IV ($2^{48} \to 2^{59}$ and $2^{15} \to 2^{27}$), because it destroys the "compare one guess with every stored hash for free" advantage. In Case III that matters a great deal: at 10^{10} hashes/s it turns 2^{48} (about 7.8 hours) into 2^{59} (about 1.8 years). In Case IV it matters little in absolute terms: the weak passwords live in a tiny search space of 2^{24} words that no salt can enlarge, so even salted the whole attack is over in 13 ms. A salt cannot make a guessable password strong. What helps is making *each guess* expensive: a deliberately slow, memory-hard hash multiplies the attacker's cost by a constant factor (10^6 here, from 13 ms to 3.7 hours) while costing the defender only one hash per login attempt, and the memory requirement removes most of the advantage of GPUs and ASICs.

(d) Evaluation of the university's policy against NIST SP 800-63B-4 [2]:

- *Minimum 8 characters.* Only acceptable if the password is part of multi-factor authentication (minimum 8); a password used alone SHALL be at least 15 characters (Sec. 3.1.1.2).

- *Capital letter, digit and symbol required.* Not allowed: verifiers SHALL NOT impose other composition rules (Sec. 3.1.1.1 and 3.1.1.2). Users answer them with predictable patterns such as `Password1!`.

- *Maximum 16 characters.* Verifiers SHOULD permit at least 64 characters, so passphrases and password-manager passwords fit (Sec. 3.1.1.2).

- *Change every 90 days.* Not allowed: verifiers SHALL NOT require periodic changes, and SHALL force a change only on evidence of compromise (Sec. 3.1.1.2). Forced rotation just produces weaker, incrementally modified passwords.

- *Missing requirements:* a blocklist check of breached, dictionary and context-specific passwords (SHALL), rate limiting of failed attempts (SHALL, at most 100 consecutive failures, Sec. 3.2.2), salted hashing with a suitable password hashing scheme and a salt of at least 32 bits (SHALL), and acceptance of paste and password managers (SHALL).

Corrected policy. (i) Password used alone: at least 15 characters, up to at least 64 (all printing ASCII, space and Unicode), no composition rules, no expiry, compare with a blocklist and reject matches, throttle guessing, store with Argon2id and a per-user salt [3], allow paste, and force a change only after a suspected compromise. (ii) Password used with a second factor: the same rules

but a minimum of 8 characters, and the second factor should preferably be phishing-resistant (the verifier SHALL offer at least one phishing-resistant option at AAL2, Sec. 2.2.2).

Question 2 — Biometrics: error rates, thresholds, and identification

All values below were computed by `q2_biometrics.py` (Appendix A.2).

(a) FAR − FRR changes sign between thresholds **50** and **60** ($+1.5 \cdot 10^{-3}$ at 50, $-1.6 \cdot 10^{-3}$ at 60), so the equal error rate lies there. With linear interpolation $\mathrm{FAR}(t) = 2 \cdot 10^{-3} - 1.6 \cdot 10^{-4}(t - 50)$ and $\mathrm{FRR}(t) = 5 \cdot 10^{-4} + 1.5 \cdot 10^{-4}(t - 50)$; setting them equal gives $t \approx 54.8$ and EER $\approx \mathbf{1.2 \cdot 10^{-3}}$. (Interpolating the logarithms instead gives $t \approx 54.6$ and EER $\approx 9.5 \cdot 10^{-4}$.) This is about 1.6 times better than the typical published fingerprint EER of $2 \cdot 10^{-3}$, so the vendor's figures look good, but vendor numbers are measured under favourable conditions and should be verified independently.

(b) The building sees $2000 \times 4 = 8000$ legitimate entries per day.

Threshold	Legitimate rejections per day	P(impostor accepted within 10 tries)
50	$8000 \cdot 5 \cdot 10^{-4} = 4$	$1 - (1 - 2 \cdot 10^{-3})^{10} \approx 1.98\%$
60	$8000 \cdot 2 \cdot 10^{-3} = 16$	$1 - (1 - 4 \cdot 10^{-4})^{10} \approx 0.40\%$
70	$8000 \cdot 10^{-2} = 80$	$1 - (1 - 5 \cdot 10^{-5})^{10} \approx 0.05\%$

I would choose threshold **70**. An impostor with ten tries succeeds with only 0.05%, and NIST SP 800-63B-4 requires a false match rate of at most one in 10 000 (Sec. 3.2.3.1), which $5 \cdot 10^{-5}$ meets and $4 \cdot 10^{-4}$ (threshold 60) does not. The price is 80 rejected entries per day (1% of all entries), but a rejected employee simply tries again: the chance of two rejections in a row is only 10^{-4}, and a badge or PIN must anyway be offered as a non-biometric alternative. A false accept is a security incident, whereas a false reject costs a few seconds.

(c) Identification compares the sample with $N = 100\,000$ templates, so P(at least one false match) $= 1-(1-\mathrm{FAR})^N \approx 1-e^{-N \cdot \mathrm{FAR}}$. At threshold 70 ($\mathrm{FAR} = 5 \cdot 10^{-5}$) the expected number of false matches is $N \cdot \mathrm{FAR} = \mathbf{5}$ and the probability of at least one is **99.3**%. At threshold 80 (10^{-6}) the expected number is 0.1 and the probability is **9.5**%. Authentication makes one comparison, identification makes N, and the errors add up: to keep the probability of any false match below 1% over 10^5 templates the FAR must be below $\approx 10^{-7}$, a thousand times smaller than what is acceptable for authentication.

(d) A breached password can be replaced, a fingerprint cannot, so a leaked template is compromised for life, and biometric traits are not secret at all (latent fingerprints, photographs) [2]. Design consequences: never keep raw templates; if templates must be stored, protect them (template protection) and keep them locally; treat the biometric as a way to *unlock* a revocable secret rather than as the secret; require presentation-attack detection; always offer a non-biometric alternative. NIST therefore allows biometrics only as part of multi-factor authentication with a physical authenticator (Sec. 3.2.3). A phone unlocking a passkey follows this design: the fingerprint is compared *on the device*, in secure hardware, and only authorizes a signature with a private key that never leaves the device; the server stores only the public key [4]. If the device is lost, the credential is revoked and re-registered, and the biometric template never reaches any server.

Question 3 — Beyond passwords: one-time codes, passkeys, and session cookies

(a) In the smartphone protocol the user needs the phone, which holds K (*something you have*), and the PIN (*something you know*), so it is two-factor authentication. R must be random and never reused: if Bob repeats a challenge, Trudy who recorded one run replays the stored response and is accepted (`q3_otp.py`, Appendix A.3: a replayed response is accepted for a reused R and rejected for a fresh one), and a predictable R would let Trudy collect responses in advance. For TOTP with $t = 1\,700\,000\,000$: $T = \lfloor 1\,700\,000\,000/30 \rfloor = \mathbf{56\,666\,666}$, and the next code appears at

$(T+1)\cdot 30 = \mathbf{1\,700\,000\,010}$, i.e. 10 seconds later. The code is computed by **HMAC** (last week's HMAC construction, RFC 2104 [5]): HOTP/TOTP take HMAC-SHA-1(K, T) and truncate it to 6 digits [6], [7]. My implementation reproduces all RFC test vectors (10 HOTP vectors and 6 TOTP vectors).

(b) A day has $86\,400/30 = 2880$ windows. With 3 attempts per window and one valid code out of 10^6, the success probability per window is $3/10^6$, so within 24 hours

$$1 - \left(1 - 3\cdot 10^{-6}\right)^{2880} \approx \mathbf{0.86}\%$$

(with no lockout the expected time to success is about 116 days). If the server also accepts the previous and next window, three codes are valid, the per-window probability becomes $9\cdot 10^{-6}$, and

$$1 - \left(1 - 9\cdot 10^{-6}\right)^{2880} \approx \mathbf{2.56}\%$$

(about 39 days on average). The attack is limited by the **rate-limiting rule of NIST SP 800-63B-4, Sec. 3.2.2**: the verifier SHALL limit consecutive failed attempts to no more than 100 and then disable the authenticator, and NIST states that the limit of 100 was chosen with a six-digit OTP in mind [2]. With 100 guesses Trudy succeeds with at most 0.01% (0.03% with clock-drift tolerance).

(c) With a real-time phishing proxy, Trudy's look-alike page shows a login form; Alice types her password and the current TOTP or SMS code; the fake page forwards both at once to the real site and receives the session. Passwords, TOTP codes and SMS codes are values that a human types, so nothing ties them to the site that receives them: NIST states that manually entered authenticator output is not phishing-resistant because it is not bound to the session being authenticated [2]. A **passkey** (FIDO2/WebAuthn) instead computes a digital signature, in the style of last week's ECC lab, normally ECDSA over P-256 (ES256) or EdDSA: the authenticator signs authenticatorData $\|$ SHA-256(clientDataJSON), where *clientDataJSON contains the challenge and the origin the browser actually sees*, and authenticatorData contains SHA-256 of the relying-party ID [4]. This defeats the relay in three independent ways (simulated in `q3_phishing.py`, Appendix A.4): (1) the credential is scoped to the real domain, so on the look-alike domain the browser finds no passkey and nothing is signed; (2) if a signature were produced anyway, the real site sees the wrong origin and relying-party hash and rejects it; (3) Trudy cannot rewrite the origin, because the signature covers the hash of the client data. Moreover the private key never leaves the authenticator, so there is no reusable secret to phish.

MFA fatigue works against push-based approvals: Trudy already has the password and triggers login attempts again and again, so Alice's phone shows "Approve sign-in?" prompts, often at night; sooner or later she taps *approve* to make them stop or because she believes a colleague asked (sometimes helped by a call from "IT support"). *Number matching* fixes this: the login screen shows a two-digit number that Alice must type into the app, so an approval can only be given by someone who sees the real login screen; limits on the number of push notifications (NIST SHOULD, Sec. 3.1.3.2) and context such as the application and the location help too.

(d) After login the site trusts a *session cookie*, a bearer token: whoever presents it is treated as the logged-in user, and the authentication factors were checked only once, at login. A stolen cookie (through cross-site scripting, malware or infostealers, an unencrypted connection, or an adversary-in-the-middle proxy that relays the MFA and keeps the cookie) therefore bypasses every factor. Settings that reduce the risk [2]: `Secure` (HTTPS only), `HttpOnly` (not readable by JavaScript), and `SameSite=Lax` or `Strict`; in addition the `__Host-` prefix, short idle and absolute lifetimes, a fresh session identifier after login, re-authentication for sensitive actions, and binding the session to the device (DBSC).

Authorization

Question 4 — Access control matrix, ACLs, capabilities, and the confused deputy

The matrix was entered into `q4_access_control.py` (Appendix A.5), which derives the ACL and the C-list and checks the matrix for the problems discussed below.

(a) Stored by column the matrix gives the ACLs, stored by row the capabilities.

- ACL of the grade database: Besnik: `rw`, Drita: `rw`, Ermal: `r`, GradingService: `rw`. (Arta: no access.)
- C-list of Besnik: ExamQuestions: `r`, SubmittedAnswers: `r`, GradeDB: `rw`, CoursePage: `rw`.

(b) Problems of the matrix:

1. *Separation of duties.* GradingService may write the grade database **and** the audit log. The program that changes grades can also rewrite the evidence of its own changes.
2. *Least privilege* (see the design principles after Question 5). Besnik, a teaching assistant, has `rw` on the final grade database. A TA should *propose* grades and the professor approve them.
3. *The matrix cannot express ownership.* Arta has `w` on the single object SubmittedAnswers, so she may overwrite **everyone's** answers, and the matrix cannot say "only my own, and only before the deadline".
4. *Missing right:* Arta has no read right on the grade database, so she cannot see her own grade; the matrix cannot say "only my own record".
5. *Audit log.* The only right is `w` (overwrite), and there is no append-only right, so the holder can alter history.

Corrected design: (1) the TA gets `rw` on a new object ProposedGrades and no access to the final grade database; the professor approves and writes final grades (two-person rule); (2) the audit log becomes **append-only** and is written through a separate logging service that GradingService can only append to, and no one holds `w` on past entries; (3) submitted answers become *one object per student*, with `w` only for the owner and only before the deadline; (4) each student gets `r` on her own grade record; (5) the registrar and the professor keep read access to the log.

(c) Arta has no access to the grade database ("—"), but the GradingService has `rw` on it. When Arta gives the path of the grade database as the name of the exported PDF, the service opens that file for writing *using its own authority*: the check is done against the subject GradingService, not against Arta, so it is allowed and the grade database is overwritten by a PDF report (simulated in Appendix A.5; a direct write by Arta is denied). The service is a **confused deputy**: a program with more rights than its caller, tricked into using them for the caller. With **capabilities** the request carries the capability to the destination, and the service writes only to the handle it is given; Arta's C-list contains only her own report file, she cannot name the grade database, and in the simulation the same request is denied (designation and authority travel together; there is no ambient authority) [8]. A modern attack with the same pattern is **server-side request forgery (SSRF)** in the cloud: the web server is tricked into sending a request, with its own cloud role, to the instance metadata service, as in the Capital One breach of Question 6; related patterns are CSRF (the browser is the deputy) and the cross-account "confused deputy" in cloud identity services.

(d) In the new semester the matrix would have $1050 \times 5 = 5250$ cells. With **role-based access control** [9] only 3 roles $\times$ 5 objects = 15 permission cells plus 1050 user–role assignments are needed: adding a student is one assignment instead of five cells, and changing a rule for all students is one edit instead of 1000. Roles also make the TA/professor separation of (b) natural. What RBAC alone cannot express is requirement (b)3–4: *per-record ownership and conditions*, such as "a student

may write only her own answers, only before the deadline" and "a student may read only her own grade". These need attribute- or relationship-based rules on top of roles. (RBAC also does not by itself prevent the confused deputy of (c).)

Question 5 — Multilevel security, covert channels, and inference

All decisions below were computed by `q5_mls.py` (Appendix A.6), which implements dominance $(L_1, C_1) \geq (L_2, C_2) \iff L_1 \geq L_2 \wedge C_1 \supseteq C_2$.

(a) Bell–LaPadula with compartments.

#	Request	Decision	Deciding rule and reason
1	Alice reads O_1 (S,{Cat})	allowed	no read up: $(\mathrm{TS}, \{Cat\})$ dominates $(\mathrm{S}, \{Cat\})$
2	Alice reads O_5 (S,{Cat,Dog})	denied	no read up: level is fine but Alice lacks the compartment Dog
3	Alice writes O_4 (U,{})	denied	no write down: O_4 is below Alice's Top Secret level
4	Bob reads O_5	allowed	no read up: identical labels, so Bob dominates O_5
5	Bob writes O_3 (C,{Dog})	denied	no write down: O_3's level Confidential is below Secret
6	Carol writes O_2 (TS,{})	allowed	*-property: O_2 dominates Carol, so writing up is allowed
7	Carol reads O_1 (S,{Cat})	denied	no read up: Carol lacks the compartment Cat
8	Dave reads O_3 (C,{Dog})	denied	no read up: same level but Dave lacks Dog

A subject may write to $O_1 = (\mathrm{S}, \{Cat\})$ only if O_1 dominates the subject's label. Alice (TS) and Bob (needs the compartment Dog) fail, so exactly **Carol (S,{}) and Dave (C,{})** may write to O_1, although neither of them may read it (blind writes up).

(b) Biba (integrity: no write up, no read down). The browser (Low) may **not** write the operating-system files (High): that would be writing up. The installer (High) may **not** read the downloaded file (Low) under strict Biba, which forbids reading down. Under the **low water mark** policy the installer *may* read the file, but its integrity level drops to $\min(\mathrm{High}, \mathrm{Low}) = \mathrm{Low}$. Afterwards it may write only objects of integrity Low, that is the downloaded file, and no longer the configuration file or the operating-system files (the script lists the allowed writes before and after). The idea is that a process that has touched untrusted data must no longer be trusted to modify trusted data. The mechanism of a modern operating system is **Windows Mandatory Integrity Control** (integrity levels low, medium, high and system): by default an object gets the `NO_WRITE_UP` policy, so a low-integrity process such as a sandboxed browser cannot write to a medium-integrity object even if the object's DACL would allow it [10].

(c) At 1 bit per second a 256-bit AES key leaks in 256 s $\approx$ **4.3 minutes**, a 3072-bit RSA private exponent in 3072 s $\approx$ **51 minutes**, and a 1 MiB document (8 388 608 bits) in 8 388 608 s $\approx$ **97 days**. A channel that is slow enough to protect bulk data is still fast enough to steal the small, high-value secrets (keys), so the bandwidth limit must be judged against what is leaked, and the real defence is to remove the shared resource or the timing signal. A real example is the cache timing channel exploited by **Spectre** [11]: the cache state is a shared resource that the attacker can vary and observe, and it carries secrets across a privilege boundary.

(d) Query 1 gives a sum of $20 \cdot 8.10 = 162.0$ and query 2 a sum of $19 \cdot 8.00 = 152.0$. The difference tells us that **exactly one student is from Gjilan and that student's final grade is** $162.0 - 152.0 = \mathbf{10.0}$. Query-set-size control failed because it examines each query on its own: both sets have at least 5 members and are answered, yet the *difference* of two permitted queries isolates a single record (a tracker or differencing attack); the direct query "students from Gjilan" would have been refused. A better defence is **differential privacy** [12]: add noise calibrated to the maximum influence of one record. With Laplace noise of scale $10/\varepsilon$ ($\varepsilon = 1$, one grade changes a sum by at most 10) the noisy difference in the simulation was 16.8 and its typical error ≈ 15 points, so the individual grade is hidden while sums over the whole class stay useful. Auditing of the query history and limits on overlapping query sets are complementary controls.

Design principles: least privilege and least authority

Two design principles run through the findings of Questions 3 to 6.

Least privilege [13], [14]: every user, program and role should hold only the rights it needs for its task, and only for as long as it needs them. It does not stop an attack, but it limits what a stolen password, a tricked program or a dishonest insider can do. Its price is administration: rights must be reviewed, otherwise privileges accumulate over time, which is one reason for roles (Question 4(d)).

Least authority (the principle of least authority, from capability systems [8]) looks at the same problem from the other side: not how small the set of rights is, but *where a program's authority comes from.* A program should act only with the specific authority it was handed for the current request, instead of an *ambient* set of rights that applies to everything it does. The confused deputy of Question 4(c) is exactly a failure of this principle: the GradingService held the ambient right to write the grade database, so any file name chosen by a user could make use of it. Passing a capability to the destination gives the service no more authority than the request needs.

Where it appears	Principle	What it requires
Q3(d) session cookie	least privilege	cookie limited to the minimum practical hostnames and paths and to a short lifetime [2]
Q4(b) TA, service and audit log	least privilege	the TA may only propose grades; GradingService may only append to the log
Q4(c) exported file name	least authority	the service writes only to the handle the user passes, never to a path it can reach by itself
Q5(b) Biba low water mark	least privilege, applied automatically	a process that has read untrusted data loses the right to write high-integrity objects
Q6 Capital One firewall role	both	the role is limited to what a firewall needs; the metadata service no longer hands the role's credentials to any request from the host

The two principles complement each other: least privilege shrinks the set of rights, and least authority removes the ambient path through which a deputy can be tricked into using them.

Real-World Investigation

Question 6 — The 2019 Capital One breach

(a) Incident profile.

Date(s)	Intrusion on 22–23 March 2019; reported to Capital One by an external researcher on 17 July 2019; confirmed 19 July; disclosed on 29 July 2019, when the suspect was arrested [15], [16].
Organisations and scale	Capital One: about 106 million people (about 100 million in the US and 6 million in Canada); about 140 000 Social Security numbers, 1 million Canadian social insurance numbers and 80 000 linked bank account numbers. Prosecutors said that data of more than 30 organisations was taken [15], [17].
Attacker	Paige A. Thompson, a former Amazon Web Services engineer (handle "erratic"); convicted of seven federal counts in June 2022 [17].
Weakness	Authorization: a misconfigured web application firewall on an AWS EC2 instance could be made to fetch credentials of its IAM role, which was allowed to list and read data in many S3 buckets (an over-broad role and a confused deputy) [16], [18].
Official sources	Criminal complaint (reported in [16]); Capital One SEC filing [15]; OCC news release 2020-101 [19].
Outcome	OCC civil penalty of 80 million USD (6 August 2020) [19]; 190 million USD class-action settlement; Capital One expected costs of 100–150 million USD in 2019 [15], [17]; AWS introduced IMDSv2 in November 2019 [20].

(b) Attack chain.

1. Capital One ran a web application firewall on an EC2 instance. A misconfiguration let the attacker make this server send requests of her choosing; this is widely analysed as server-side request forgery (SSRF) against the instance metadata service [18] (the complaint itself does not state the method [16]).

2. At that time the metadata service (IMDSv1) answered any plain HTTP request made from the instance with temporary credentials of the IAM role attached to it, and it authenticated nothing [20].

3. The role of the firewall (named `*****-WAF-Role` in the complaint) was permitted to list and read data in more than 700 S3 folders and buckets, far more than a firewall needs [16].

4. With these valid, temporary credentials, from outside the company, she listed and copied the data, which contained credit-card applications: names, addresses, dates of birth and, for some, Social Security numbers.

5. She boasted about it online; a tip reached Capital One on 17 July 2019, almost four months after the intrusion [15].

No password, factor or biometric was guessed or phished. What was **too broad was an authorization rule** (the role's permissions), and the credentials were bearer tokens that anyone who held them could use.

(c) Classification. It is a failure of **authorization**, not of authentication: (1) a violation of **least privilege** (the firewall role could read customer data); (2) a **confused deputy**, the same pattern as the GradingService in Question 4(c): a server with its own privileges was tricked by an outsider into using them on her behalf; and (3) the stolen temporary credentials behaved like a stolen **session cookie** (Question 3(d)): once held, they bypassed all authentication. Multi-factor authentication, passkeys and biometrics would not have helped, because no human login was involved.

(d) Response and lessons. Capital One fixed the misconfiguration, notified the affected customers and offered credit monitoring. The OCC found that the banks had failed to establish effective risk assessment processes before moving significant IT operations to the public cloud and to correct deficiencies in a timely manner, and imposed 80 million USD [19]; a class action ended with 190 million USD [17]. AWS added IMDSv2, which requires a session token obtained with a separate request, so that simple SSRF bugs can no longer read the credentials [20]. The technical response was substantial, but detection was slow: the company learned of the breach only from an outside tip. *Control that would have prevented or limited it:* give each cloud role the **minimum permissions** (the firewall role could be restricted to its own logging bucket, so even stolen credentials would have been useless for customer data) and **enforce IMDSv2** on all instances; alerts on unusual S3 listing and bulk reads from a role would also have shortened the four months before detection.

Conclusion

Authentication and authorization decide who gets in and what they can do, and the incidents of this lab show that they fail through people and configuration far more often than through mathematics. The password calculations showed that a salt only removes the attacker's parallelism, while a slow memory-hard hash multiplies every guess and the best defence is not letting users pick guessable passwords at all. Biometrics trade convenience against irrevocability, and identification against large databases needs error rates a thousand times lower than authentication. One-time codes and push approvals can be relayed or fatigued, whereas passkeys bind a signature to the website's origin and so resist phishing, but a stolen session cookie bypasses every factor. On the authorization side, access control matrices are static and cannot express ownership, time or separation of duties, roles make them manageable, and capabilities remove the ambient authority that makes a confused deputy

possible. Multilevel models give precise rules (Bell–LaPadula for confidentiality, Biba for integrity) but leave covert channels and inference, as the one-grade leak and the 97-day document showed. Least privilege and least authority tie the authorization findings together: the first shrinks the rights a subject holds and the second removes the ambient authority through which a deputy is confused. The Capital One breach combined an SSRF-style confused deputy with an over-broad role, and the right controls were least privilege and a metadata service that demands a token, not stronger passwords.

Appendix A — Python Verification: Chosen Values and Outputs

The scripts are in the repository https://github.com/Diellart-Mulolli/auth_lab (see its README for the running guide). They were run with Python 3.12.3 and the `cryptography` package 41.0.7 (only `q3_phishing.py` needs it). The outputs below are copied unchanged from the terminal. Key results were checked independently: the HOTP/TOTP implementation against the 16 test vectors of RFC 4226 and RFC 6238, the closed-form work formulas of Question 1 against a Monte Carlo simulation, and the numbers of Question 2 and Question 5 against hand calculations.

A.1 `q1_password_cracking.py` (Question 1)

Chosen values: $N = 64^{10} = 2^{60}$ passwords, $U = 2^{12}$ users, dictionary $D = 2^{24}$, dictionary probability $p = 1/8$, attacker speeds 10^{10} and 10^4 hashes/s; the Monte Carlo check uses $N = 2^{20}$, $U = 2^8$, $D = 2^{12}$, $p = 1/8$ and 20000 trials (fixed seed 7). Command: `python3 q1_password_cracking.py`

```
N = 64^10 = 2^60, users U = 2^12, dictionary D = 2^24, p = 1/8

case                                          unsalted                  salted
I   Alice, no dictionary             5.765e+17 = 2^59.00      5.765e+17 = 2^59.00
II  Alice, with dictionary           5.044e+17 = 2^58.81      5.044e+17 = 2^58.81
III any user, no dictionary          2.814e+14 = 2^48.00      5.765e+17 = 2^59.00
IV  any user, with dictionary         3.27e+04 = 2^15.00      1.342e+08 = 2^27.00

Case II dictionary phase alone costs 1.573e+07 = 2^23.91 hashes and succeeds only with
    probability 0.125
Case IV salted: exact expectation 1.258e+08 = 2^26.91, rule of thumb D/p = 2^27

(b) Case IV, salted
  fast, 1e10 hash/s  D/p = 2^27: 1.342e+08 / 1e+10 = 0.01342 s = 13.42 ms
  fast, 1e10 hash/s  exact     : 1.258e+08 / 1e+10 = 0.01258 s = 12.58 ms
  slow, 1e4 hash/s   D/p = 2^27: 1.342e+08 / 1e+04 = 1.342e+04 s = 3.73 hours
  slow, 1e4 hash/s   exact     : 1.258e+08 / 1e+04 = 1.258e+04 s = 3.50 hours
  slowdown from the slow hash: 1e+06x

Monte Carlo check of the closed forms (scaled-down model, 20000 trials each)
  III unsalted  N/(U+1)      formula    4080.1  simulated    4072.1  ratio 0.998
  IV  unsalted  D/(U*p+1)    formula     124.1  simulated     126.2  ratio 1.017
  IV  salted    (1-p)/p*D+D/2 formula   30720.0  simulated   30573.3  ratio 0.995
```

Check. The table reproduces Question 1(a): 2^{59}, $\approx 2^{59}$, $2^{48}/2^{59}$ and $2^{15}/2^{27}$; the times of Question 1(b) are 13.4 ms and 3.7 hours; and the simulation agrees with the formulas within 2%.

A.2 `q2_biometrics.py` (Question 2)

Chosen values: the vendor table (thresholds 40–80 with the given FAR and FRR), 2000 employees with 4 entries per day, 10 impostor tries, $N = 100\,000$ templates. Command: `python3 q2_biometrics.py`

```
(a) equal error rate
  threshold   FAR        FRR        FAR - FRR
      40   1e-02   1e-04   +9.90e-03
      50   2e-03   5e-04   +1.50e-03
      60   4e-04   2e-03   -1.60e-03
      70   5e-05   1e-02   -9.95e-03
      80   1e-06   5e-02   -5.00e-02
  FAR - FRR changes sign between thresholds 50 and 60
  linear interpolation: FAR(t) = 2e-03 + (-1.600e-04)(t-50), FRR(t) = 5e-04 + (1.500e-04)(t-50)
  crossing at t = 54.84, EER = 1.2258e-03
  cross-check, log-linear interpolation: t = 54.63, EER = 9.4968e-04
  typical published EER 2e-3: this reader is 1.63x better (linear estimate)

(b) building with 2,000 employees, 4 entries per day
  legitimate entries per day = 8000
  threshold 50: rejected per day = 8000 x 5e-04 = 4.0   (one rejection per 2000 entries)
  threshold 60: rejected per day = 8000 x 2e-03 = 16.0   (one rejection per 500 entries)
  threshold 70: rejected per day = 8000 x 1e-02 = 80.0   (one rejection per 100 entries)
  threshold 50: P(impostor accepted within 10 tries) = 1-(1-2e-03)^10 = 1.9821%
  threshold 60: P(impostor accepted within 10 tries) = 1-(1-4e-04)^10 = 0.3993%
  threshold 70: P(impostor accepted within 10 tries) = 1-(1-5e-05)^10 = 0.0500%

(c) identification against N = 100,000 templates
  threshold 70: expected false matches N*FAR = 5; P(at least one) exact = 0.99326, approx
      1-exp(-N*FAR) = 0.99326
  threshold 80: expected false matches N*FAR = 0.1; P(at least one) exact = 0.09516, approx
      1-exp(-N*FAR) = 0.09516
  FAR needed for P(false match) <= 1% over N: 1.01e-07 (vs 1:1 authentication FAR of 1e-4..1e-3)
```

Check. The sign change of FAR−FRR between 50 and 60, $t \approx 54.8$ and EER $\approx 1.2 \cdot 10^{-3}$ (log-linear cross-check $9.5 \cdot 10^{-4}$), the 4/16/80 rejections per day, the impostor probabilities, and the identification results (5 and 0.1 expected false matches; 99.3% and 9.5%) match Question 2.

A.3 `q3_otp.py` (Question 3(a),(b))

Chosen values: the RFC test key `"12345678901234567890"`; $t = 1\,700\,000\,000$, 30-second step, 6 digits; 3 attempts per window; 2880 windows per day; a random 20-byte key for the replay demonstration. Command: `python3 q3_otp.py`

```
RFC 4226 Appendix D, HOTP counters 0..9 (key '12345678901234567890'): all 10 vectors match
  computed: 755224 287082 359152 969429 338314 254676 287922 162583 399871 520489
RFC 6238 Appendix B, TOTP-SHA1, 8 digits: all 6 vectors match
  t =          59  T =         1  code = 94287082
  t =  1111111109  T =  37037036  code = 07081804
  t =  1111111111  T =  37037037  code = 14050471
  t =  1234567890  T =  41152263  code = 89005924
  t =  2000000000  T =  66666666  code = 69279037
  t = 20000000000  T = 666666666  code = 65353130

(a) challenge-response: h(K, R), K shared with Bob, R chosen by Bob
  Bob sends random R1; Alice's phone answers; Trudy records the pair (R1, response)
  later, Bob again sends R1 (reused):   Trudy replays -> accepted: True
  later, Bob sends a fresh R2:          Trudy replays -> accepted: False

TOTP counter and time of the next code
  t = 1700000000: T = floor(1700000000/30) = 56666666   (exact quotient 56666666.6667)
  the next code appears at (T+1)*30 = 56666667 * 30 = 1700000010, i.e. 10 s later
  example 6-digit TOTP at t=1700000000 with the RFC test key: 921300  (and 732303 at
      t=1700000010)

(b) Trudy knows the password and guesses 6-digit TOTP codes (3 attempts per 30 s, no lockout)
  windows in 24 h = 86400/30 = 2880
  current window only               : P(success per window) = 3.0000e-06; P(success in 24 h) =
      1-(1-3.000e-06)^2880 = 0.8603%; expected time to success = 116 days
  previous + current + next window  : P(success per window) = 9.0000e-06; P(success in 24 h) =
      1-(1-9.000e-06)^2880 = 2.5587%; expected time to success = 39 days

  NIST SP 800-63B-4, Sec. 3.2.2: at most 100 consecutive failed attempts, then the authenticator
      is disabled
  current window only               : 100 guesses -> P(success) = 0.0100%
  previous + current + next window  : 100 guesses -> P(success) = 0.0300%
```

Check. All 10 HOTP vectors (RFC 4226, Appendix D) and all 6 TOTP vectors (RFC 6238, Appendix B) match, which validates the implementation. $T = 56\,666\,666$ and the next code at $1\,700\,000\,010$ match Question 3(a); the probabilities 0.86% and 2.56% and the 100-attempt limit match Question 3(b). The two example codes depend only on the public RFC key.

A.4 `q3_phishing.py` (Question 3(c))

Chosen values: real site `https://bank.example` (relying-party ID `bank.example`), look-alike site `https://bank-example.evil`; passkey = ECDSA P-256 with SHA-256 over authenticatorData ‖ SHA-256(clientDataJSON); TOTP time 1 700 000 000. Command: `python3 q3_phishing.py`

```
=== 1. password + TOTP, phished by a real-time relay ===
  Alice types password and the 6-digit code on the FAKE site https://bank-example.evil
  the fake site instantly forwards both to the real site (inside the 30 s window)
  real site accepts the relayed login: True

=== 2. passkey, honest case (Alice on the real site) ===
  real site: ACCEPTED

=== 3. passkey, Alice is on the FAKE site, Trudy relays the real challenge ===
  3a. browser asks the authenticator for rpId = bank-example.evil
      authenticator returns a signature: False (no credential exists for that site)
  3b. worst case: an authenticator signs for the fake site anyway
      Trudy relays it to the real site -> rejected: origin is https://bank-example.evil,
          expected https://bank.example
  3c. Trudy edits the origin to the real one to get past the origin check
      real site -> rejected: signature does not verify
```

Check. The relayed password and TOTP code are accepted, while the passkey relay fails at each layer: no credential for the fake domain (3a), wrong origin and relying-party hash (3b), and an invalid signature as soon as the origin is rewritten (3c). The output contains only booleans and fixed messages, so it is identical on every run.

A.5 `q4_access_control.py` (Question 4)

Chosen values: the matrix of the lab handout (5 subjects, 5 objects); the object behind the path `/db/grades.db` is GradeDB; Arta holds one capability, her own report file; the new semester has $1000 + 20 + 30$ users. Command: `python3 q4_access_control.py`

```
access control matrix (r = read, w = write, - = none)
                     ExamQ   Answers   GradeDB  AuditLog    Course
Arta                    -         w         -         -         r
Besnik                  r         r        rw         -        rw
Drita                  rw         r        rw         r        rw
Ermal                   -         -         r         r         r
GradingService          -         r        rw         w         -

(a) ACL of GradeDB (matrix stored by COLUMN)
  GradeDB: Besnik: rw, Drita: rw, Ermal: r, GradingService: rw

C-list of Besnik (matrix stored by ROW)
  Besnik: ExamQuestions: r, SubmittedAnswers: r, GradeDB: rw, CoursePage: rw

(b) automatic checks of the matrix
  1. GradingService may write GradeDB AND AuditLog: it can change grades and rewrite the
      evidence
  2. Besnik may write GradeDB although the role (TA) only needs to propose grades: no separation
      between grading and approval
  3. Arta has w on the single object SubmittedAnswers: she can overwrite EVERYONE's answers, and
      the matrix cannot say 'only my own, only before the deadline'
  4. Arta has no read right on GradeDB, so she cannot even see her own grade (the matrix cannot
      express 'only my own record')
  5. the log has only 'w' (overwrite) and it is held by ['GradingService']; there is no
      append-only right

(c) confused deputy: GradingService exports a PDF to a file name chosen by the user
  direct attack:    DENIED  Arta may not write GradeDB
  via the service:  allowed GradingService wrote GradeDB
  grade database now contains: 'PDF report requested by Arta'
  capability version, Arta passes her own report capability: allowed wrote /reports/arta.pdf
  capability version, Arta asks for the grade database:      DENIED  Arta holds no capability
      named 'grade_db'
  grade database now contains: 'grade records for 1000 students'

(d) RBAC for next semester
  users = 1000 + 20 + 30 = 1050
  matrix cells (users x 5 objects) = 5250
  RBAC: 3 roles x 5 objects = 15 permission cells + 1050 user-role assignments
  adding one student: 1 assignment (instead of 5 cells); changing a rule for all students: 1
      edit (instead of 1000)
```

Check. The ACL and the C-list are those of Question 4(a), the five findings are those of 4(b), and the simulation reproduces 4(c): a direct write by Arta is denied but the same write through GradingService succeeds and destroys the grade database, while the capability version refuses the request. The RBAC counts (5250 versus 15 cells plus 1050 assignments) match 4(d).

A.6 `q5_mls.py` (Question 5)

Chosen values: the labels of the four subjects and five objects of the lab handout; Biba levels Low $<$ Medium $<$ High; a covert channel of 1 bit/s; two inference queries (20 students, average 8.10; 19 students, average 8.00); Laplace noise of scale $10/\varepsilon$ with $\varepsilon = 1$ (seed 2026). Command: `python3 q5_mls.py`

```
(a) Bell-LaPadula with compartments
  subject Alice  (Top Secret, {Cat})
  subject Bob    (Secret, {Cat, Dog})
  subject Carol  (Secret, {})
  subject Dave   (Confidential, {})
  object  O1     (Secret, {Cat})
  object  O2     (Top Secret, {})
  object  O3     (Confidential, {Dog})
  object  O4     (Unclassified, {})
  object  O5     (Secret, {Cat, Dog})
  requests:
   (1) Alice read  O1: ALLOWED  simple security property (no read up); subject dominates object
   (2) Alice read  O5: DENIED   simple security property (no read up); subject lacks
       compartment(s) ['Dog']
   (3) Alice write O4: DENIED   *-property (no write down); object level is lower
   (4) Bob   read  O5: ALLOWED  simple security property (no read up); subject dominates object
   (5) Bob   write O3: DENIED   *-property (no write down); object level is lower
   (6) Carol write O2: ALLOWED  *-property (no write down); object dominates subject
   (7) Carol read  O1: DENIED   simple security property (no read up); subject lacks
       compartment(s) ['Cat']
   (8) Dave  read  O3: DENIED   simple security property (no read up); subject lacks
       compartment(s) ['Dog']
  subjects that may write to O1 (Secret, {Cat}): ['Carol', 'Dave']
  of these, who may also read O1: []  (blind write-up)

(b) Biba integrity
  browser (Low) writes OS files (High):       False
  installer (High) reads downloaded file (Low): False  (strict Biba: no read down)
  low water mark: installer reads the downloaded file
    I(installer) := min(High, Low) = Low
    before: installer may write ['OS files', 'config file', 'downloaded file']
    after : installer may write ['downloaded file']

(c) covert channel at the DoD limit of 1 bit/s
  256-bit AES key                 :       256 bits -> 256 s = 4.3 min
  3072-bit RSA private exponent   :      3072 bits -> 3072 s = 51.2 min
  1 MiB document                  :   8388608 bits -> 97.1 days

(d) inference against 'at least 5 records' query-set-size control
  Query 1: 20 students, average 8.1  -> sum = 162.0
  Query 2: 19 students, average 8.0  -> sum = 152.0
  both sets have at least 5 members, so both are answered
  difference: 20 - 19 = 1 student(s) from Gjilan, with grade 162.0 - 152.0 = 10.0
  asked directly, 'students from Gjilan' (size 1) would have been refused: 1 < 5

better defence: add calibrated Laplace noise (differential privacy, epsilon = 1)
  noisy sums: 156.3 and 139.5; noisy difference = 16.8 (true value 10.0)
  typical error of the difference over 20000 repetitions: 15.0 grade points (the grade itself is
      at most 10), so the individual grade is hidden
```

Check. The eight Bell–LaPadula decisions and the writers of O_1 (Carol and Dave) match Question 5(a); the Biba results match 5(b); the leak times (4.3 minutes, 51.2 minutes, 97.1 days) match 5(c); and the sums 162.0 and 152.0 give the single grade 10.0 of 5(d).

References

[1] M. Stamp, *Information Security: Principles and Practice*, 3rd ed. Hoboken, NJ, USA: Wiley, 2022, chs. 6–7.

[2] D. Temoshok *et al.*, "Digital identity guidelines: Authentication and authenticator management," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST SP 800-63B-4, Jul. 2025, doi: 10.6028/NIST.SP.800-63B-4.

[3] OWASP, "Password storage cheat sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (accessed Oct. 27, 2026).

[4] World Wide Web Consortium, "Web authentication: An API for accessing public key credentials, Level 2," W3C Recommendation, Apr. 2021. [Online]. Available: https://www.w3.org/TR/webauthn-2/

[5] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," Internet Eng. Task Force, RFC 2104, Feb. 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

[6] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache, and O. Ranen, "HOTP: An HMAC-based one-time password algorithm," Internet Eng. Task Force, RFC 4226, Dec. 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4226

[7] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-based one-time password algorithm," Internet Eng. Task Force, RFC 6238, May 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6238

[8] N. Hardy, "The confused deputy: (or why capabilities might have been invented)," *ACM SIGOPS Oper. Syst. Rev.*, vol. 22, no. 4, pp. 36–38, Oct. 1988.

[9] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, Aug. 2001.

[10] Microsoft, "Mandatory integrity control," Microsoft Learn, Win32 apps. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/secauthz/mandatory-integrity-control (accessed Oct. 27, 2026).

[11] P. Kocher *et al.*, "Spectre attacks: Exploiting speculative execution," in *Proc. 40th IEEE Symp. Security Privacy (S&P)*, San Francisco, CA, USA, 2019, pp. 1–19.

[12] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of Cryptography (TCC 2006)* (LNCS 3876). Berlin, Germany: Springer, 2006, pp. 265–284.

[13] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proc. IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975, doi: 10.1109/PROC.1975.9939.

[14] Joint Task Force, "Security and privacy controls for information systems and organizations," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST SP 800-53 Rev. 5, Sep. 2020, control AC-6 (Least privilege), doi: 10.6028/NIST.SP.800-53r5.

[15] Capital One Financial Corp., "Form 10-Q for the quarterly period ended June 30, 2019," U.S. Securities and Exchange Commission, 2019. [Online]. Available: https://www.sec.gov/Archives/edgar/data/927628/000092762819000274/cof-06302019x10q.htm (accessed Oct. 27, 2026).

[16] TechTarget, "FBI charges former AWS engineer in Capital One breach," *SearchSecurity*, Jul. 2019. [Online]. Available: https://www.techtarget.com/searchsecurity/news/252467616/FBI-charges-former-AWS-engineer-in-Capital-One-breach (accessed Oct. 27, 2026).

[17] CNBC, "Former Amazon employee convicted in Capital One hack," Jun. 18, 2022. [Online]. Available: https://www.cnbc.com/2022/06/18/former-amazon-employee-convicted-in-capital-one-hack.html (accessed Oct. 27, 2026).

[18] TechTarget, "Capital One hack highlights SSRF concerns for AWS," *SearchSecurity*, Aug. 2019. [Online]. Available: https://www.techtarget.com/searchsecurity/news/252467901/Capital-One-hack-highlights-SSRF-concerns-for-AWS (accessed Oct. 27, 2026).

[19] Office of the Comptroller of the Currency, "OCC assesses $80 million civil money penalty against Capital One," News Release 2020-101, Aug. 6, 2020. [Online]. Available: https://www.occ.gov/news-issuances/news-releases/2020/nr-occ-2020-101.html (accessed Oct. 27, 2026).

[20] C. MacCárthaigh, "Add defense in depth against open firewalls, reverse proxies, and SSRF vulnerabilities with enhancements to the EC2 instance metadata service," AWS Security Blog, Nov. 2019. [Online]. Available: https://aws.amazon.com/blogs/security/defense-in-depth-open-firewalls-reverse-proxies-ssrf-vulnerabilities-ec2-instance-metadata-service/ (accessed Oct. 27, 2026).